

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

INGENIERÍA INFORMÁTICA · INF781 — SEGURIDAD DE SOFTWARE

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

PRÁCTICA N° 4

Implementación de un mecanismo CAPTCHA alternativo en Laravel 13

Campo	Detalle
Materia	INF781 — Seguridad de Software
Vinculada a	Guía de Laboratorio N° 6 (CAPTCHA en Laravel 13)
Modalidad	Individual
Tipo	Implementación práctica
Duración estimada	6 a 8 horas de trabajo autónomo
Plazo de entrega	Una semana a partir de la asignación
Entregables	Repositorio GitHub + README
Ponderación	100 puntos

1. Contexto

En la Guía de Laboratorio 6 implementaste dos mecanismos CAPTCHA: reCAPTCHA v2 de Google (servicio de tercero con widget visible) y mews/captcha (solución local basada en imágenes con texto distorsionado). Ambos representan enfoques tradicionales, pero el ecosistema actual ofrece alternativas con compromisos distintos en privacidad, accesibilidad y resistencia a IA.

En esta tarea vas a elegir UNO de los mecanismos alternativos sugeridos e integrarlo en un proyecto Laravel 13 funcional. La meta es estrictamente práctica: que demuestres que sabes evaluar opciones, leer documentación de un proveedor o paquete nuevo, y llevarlo a producción de forma segura, especialmente con verificación correcta del lado servidor.

2. Objetivos

2.1 Objetivo general

Implementar un mecanismo CAPTCHA alternativo, distinto a los vistos en clase, dentro de una aplicación Laravel 13, aplicando correctamente los principios de verificación del lado servidor, separación de secretos y validación con pruebas automatizadas.

2.2 Objetivos específicos

- Elegir un mecanismo CAPTCHA alternativo de la lista propuesta y leer su documentación oficial.
- Construir una aplicación Laravel 13 limpia con un formulario público que se beneficie de protección anti-bot.
- Integrar el mecanismo elegido con verificación correcta server-side.
- Documentar paso a paso el proceso de instalación, configuración y uso en un README reproducible.

3. Mecanismos disponibles para elegir

Debés elegir exactamente UN mecanismo de la siguiente lista. Para evitar saturación, no más de tres estudiantes pueden elegir el mismo (orden de inscripción ante el docente vía mensaje).

Opción A · Cloudflare Turnstile

Reemplazo moderno de reCAPTCHA, gratuito y respetuoso con la privacidad: no usa cookies de seguimiento. Se integra con un widget JavaScript que emite un token a verificar contra el endpoint challenges.cloudflare.com/turnstile/v0/siteverify.

Documentación: <https://developers.cloudflare.com/turnstile/>

Opción B · hCaptcha

Alternativa comercial a reCAPTCHA con API casi idéntica. Modelo de negocio basado en monetización de datasets. Tiene plan gratuito generoso para sitios pequeños. Verificación contra <https://api.hcaptcha.com/siteverify>.

Documentación: <https://docs.hcaptcha.com>

Opción C · Friendly Captcha

Proof-of-work: el navegador del cliente resuelve un problema criptográfico costoso antes de enviar el formulario. Sin desafíos visuales y completamente accesible. Tiene plan gratuito con marca y planes pagos sin marca.

Documentación: <https://docs.friendlycaptcha.com>

Opción D · reCAPTCHA v3 con scoring adaptativo

Versión invisible de reCAPTCHA: en lugar de un desafío visible, devuelve un score de riesgo entre 0.0 y 1.0. Tu servidor decide la acción según ese score (aceptar, pedir verificación adicional o rechazar). Requiere construir lógica de decisión propia.

Documentación: <https://developers.google.com/recaptcha/docs/v3>

Opción E · Honeypot avanzado + análisis de tiempo

Sin servicio externo: implementación propia que combina un campo honeypot oculto, un time-trap (rechazar formularios enviados antes de 2 segundos desde su carga), y validación de campos invisibles adicionales. Apropiado como ejercicio de implementación nativa.

Sin servicio externo. Referencia: <https://laravel-livewire.com/screencasts/honeypot> y patrones genéricos OWASP.

Restricciones

No es válido elegir reCAPTCHA v2 ni mews/captcha, dado que ya los implementaste en la G6. Tampoco copiar literalmente código de tutoriales online: la verificación del docente incluye preguntar por las decisiones de implementación.

4. Especificación del prototipo

4.1 Stack obligatorio

- Laravel 13 con PHP 8.3 o superior.
- PostgreSQL como base de datos.
- Breeze + Blade para autenticación base.
- Git con repositorio público en GitHub.

4.2 Requisitos funcionales

1. Aplicación Laravel 13 limpia con al menos un formulario público sensible (registro, login o contacto).
2. Integración del mecanismo CAPTCHA elegido en ese formulario.
3. Verificación obligatoriamente del lado servidor: si el cliente envía un token inválido o ausente, el formulario debe rechazarlo.
4. Manejo correcto de errores con mensajes claros para el usuario.
5. Configuración de claves o secretos del mecanismo exclusivamente en .env, nunca en el código.

4.3 Estructura sugerida del repositorio

estructura

```
INF781-Tarea4-<Apellido>/
├── app/
│   ├── Http/Controllers/
│   └── Rules/
│       └── <NombreMecanismo>Rule.php
├── routes/
├── resources/views/
├── tests/
│   ├── Feature/
│   └── CaptchaAlternativeTest.php
├── docs/
│   └── screenshots/           # capturas del prototipo
├── .env.example               # plantilla SIN secretos reales
├── README.md
└── composer.json
```

Seguridad de los secretos

Las claves de Turnstile, hCaptcha, Friendly Captcha o reCAPTCHA v3 son secretos. Nunca deben aparecer en commits, capturas ni en el repositorio público. Solo subir .env.example con valores placeholder. Si por error subís una clave real, debés rotarla inmediatamente en el panel del proveedor.

5. Contenido obligatorio del README

El README.md del repositorio debe contener al menos las siguientes secciones:

1. Título y descripción del proyecto (qué mecanismo elegiste y por qué).
2. Requisitos previos (PHP, Composer, Node, PostgreSQL, cuenta del proveedor si aplica).
3. Pasos para obtener las claves del proveedor (capturas o enlaces oficiales).
4. Instrucciones de instalación reproducibles desde un clon limpio: clone, composer install, npm install, .env, migrate, serve.
5. Cómo ejecutar la aplicación localmente y dónde ver el formulario protegido.
6. Cómo ejecutar las pruebas: comando exacto y resultado esperado.
7. Sección "Decisiones de diseño": en 3 a 5 párrafos, explicá por qué elegiste este mecanismo, qué te gustó y qué dificultades encontraste.
8. Capturas de pantalla del formulario funcionando: con verificación correcta y con verificación fallida.
9. Licencia y autor.

6. Entregables

#	Entregable	Formato
1	Repositorio público en GitHub	INF781-Tarea4-<Apellido>
2	README.md completo	Con las 9 secciones obligatorias
3	Sustentación oral en horario de clases	5 minutos demo + preguntas

6.1 Forma de entrega

1. Crear el repositorio público en GitHub con el nombre indicado.
2. Enviar el enlace del repositorio al docente por la vía habitual antes del plazo.
3. El último commit antes del plazo es el evaluado.
4. La sustentación se realiza en aula en la fecha asignada por el docente.

7. Rúbrica de evaluación

La calificación se distribuye sobre 100 puntos. Se requiere un mínimo de 60 para aprobar la tarea.

Criterio	Indicador	Pts
Proyecto base funcional	Laravel 13 + PostgreSQL + Breeze instalado y operativo	10
Integración del mecanismo	Widget o lógica del mecanismo elegido funcionando en el formulario	25
Verificación server-side	Validación correcta en el backend, no solo en el cliente	20
Manejo de errores y secretos	Mensajes claros, .env.example, sin secretos en el repo	10
Pruebas PHPUnit	Mínimo 2 tests pasando, cubren aceptación y rechazo	12
README y documentación	Reproducibile, decisiones de diseño explicadas, capturas	10
Calidad de código	Convenciones PSR-12, regla separada, controlador limpio	5
Sustentación oral	Demuestra dominio del mecanismo elegido, responde con solvencia	8
TOTAL		100

8. Recursos sugeridos para empezar

- Documentación oficial de Laravel 13: <https://laravel.com/docs/13.x>
- Laravel HTTP Client: <https://laravel.com/docs/13.x/http-client>
- Laravel Custom Validation Rules: <https://laravel.com/docs/13.x/validation#custom-validation-rules>
- Cloudflare Turnstile: <https://developers.cloudflare.com/turnstile/>
- hCaptcha: <https://docs.hcaptcha.com/>
- Friendly Captcha: <https://docs.friendlycaptcha.com/>
- reCAPTCHA v3: <https://developers.google.com/recaptcha/docs/v3>
- OWASP — Automated Threats to Web Applications: <https://owasp.org/www-project-automated-threats-to-web-applications/>

Recomendación final

Esta tarea es estrictamente práctica. No se pide ensayo ni paper. Se pide que demuestres saber leer una documentación nueva, integrarla limpiamente en Laravel y aplicar la lección más importante de la unidad: la verificación SIEMPRE es del lado servidor. En la sustentación oral te voy a pedir que abras el archivo donde está esa verificación y me la expliques línea por línea.

— FIN DE LA PRÁCTICA N° 4 —

INF781 — Seguridad de Software · UATF · Ingeniería Informática